

Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: IMPLEMENTACIÓN DE ANALIZADORES LÉXICOS EN LEX.

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Resumen

En este ejercicio se implementa un analizador léxico diseñado para contabilizar ocurrencias específicas de palabras, diferenciando entre la palabra independiente "el" y el pronombre "ella".

2. Estructura de un Archivo LEX

Un archivo fuente lex se encuentra estructurado por los siguientes bloques funcionales separados por (%%):

- **Área de declaración de variables:** Se declaran las librerías y variables que se van a utilizar.
- **Área de expresiones regulares:** En esta se adjuntan todas las expresiones regulares necesarias para cada programa y su respectiva impresión en consola si se llega a encontrar algo relacionado con la misma.
- **Área de código:** En esta se empieza a codificar en el lenguaje previamente seleccionado.

```
iccd332@DESKTOP-52QQFVV x + v
(base) iccd332@DESKTOP-52QQFVV:~$ mkdir diez
(base) iccd332@DESKTOP-52QQFVV:~$ cd diez
(base) iccd332@DESKTOP-52QQFVV:~/diez$ nano diez.l
(base) iccd332@DESKTOP-52QQFVV:~/diez$ nano entrada.txt
(base) iccd332@DESKTOP-52QQFVV:~/diez$ lex diez.l
(base) iccd332@DESKTOP-52QQFVV:~/diez$ gcc lex.yy.c -lfl
(base) iccd332@DESKTOP-52QQFVV:~/diez$ ls
a.out diez.l entrada.txt lex.yy.c
(base) iccd332@DESKTOP-52QQFVV:~/diez$ ./a.out < entrada.txt > salida.txt
(base) iccd332@DESKTOP-52QQFVV:~/diez$ cat salida.txt
Numero de el independientes 4
Numero de el en ellas 3
(base) iccd332@DESKTOP-52QQFVV:~/diez$ |
```

```
iccd332@DESKTOP-52QQFVV × + v
GNU nano 7.2 diez.l *
%{
    int num_el=0, num_ella=0;
}%
%%
el      num_el++;
ella    num_ella++;
\n      |
.        ;
%%
int main()
{
    yylex();
    printf("Numero de el independientes %d\n",num_el);
    printf("Numero de el en ellas %d\n",num_ella);
    return 0;
}
```

```
iccd332@DESKTOP-52QQFVV × + v
GNU nano 7.2 entrada.txt *
el perro caminaba con ella
ella le dijo a el que el gato es de ella
el
```

3. Conclusión

La ejecución de este programa permite concluir que el motor de expresiones regulares de Lex es capaz de resolver solapamientos de cadenas de manera nativa sin necesidad de lógica condicional extra en C.